

CodeSentinel AI

Revisor de Código Local con IA

Linda Valentina Lopez Rubiano

Juan Felipe Andrade

Mayo 2026

Contents

Project Report — CodeSentinel AI	3
1. Introduction	3
2. Problem statement	3
3. Objectives	3
4. Motivation	4
5. State of the art (May 2026)	4
6. Solution overview	4
7. Results preview	5
8. Scope and constraints	5
9. Document map	5
Requirements	6
Functional requirements	6
Non-functional requirements	8
Out of scope (this iteration)	8
Architecture	9
System diagram	9
Review sequence	10
Package responsibilities	11
Technology choices and trade-offs	12
Data flow boundaries	12
Model Card — Qwen2.5-Coder 7B Instruct (Q4_K_M)	13
Model details	13
Why this model	13
Models considered	13
Intended use	13
Out-of-scope use	14
Known biases and failure modes	14
Performance characteristics (measured on M5, 16 GB RAM)	14
Environmental impact	14
Ethical considerations	14
Prompt engineering	15
Design goals	15
The system prompt	15
Few-shot examples	15
Augmentation block	15
Robust JSON extraction	16

Generation parameters	16
Token budget	17
Testing	17
Strategy	17
Test inventory	18
Coverage	18
Manual UI checklist	19
How to run	19
Results — review of bundled fixtures	20
Run conditions	20
Aggregate	20
examples/buggy_python.py — planted vs. detected	21
examples/insecure_js.js — planted vs. detected	22
examples/messy_typescript.ts — planted vs. detected	22
examples/legacy_java.java — planted vs. detected	22
False-positive rate	22
Discussion	23
Limitations	23
LLM-related	23
Engineering	23
Methodological	24
Operational	24
Future work	24
Near term (1–2 weeks each)	24
Medium term (1–3 months)	24
Long term	25
Presentation script — 10-minute live demo	25
0:00 — Intro (60 s)	25
1:00 — The problem (45 s)	25
1:45 — Architecture in 30 seconds (60 s)	26
2:45 — Demo: CLI on a buggy file (2 min 30 s)	26
5:15 — Demo: web UI (2 min)	26
7:15 — Show robustness (60 s)	26
8:15 — Tests + coverage (45 s)	27
9:00 — Limitations + future (60 s)	27
10:00 — Close (30 s)	27
Backup answers (likely Q&A)	27
Run book — troubleshooting	28
“ollama: command not found”	28
“Ollama daemon not reachable on 127.0.0.1:11434”	28
“model ‘qwen2.5-coder:7b-instruct-q4_K_M’ not found”	28
“python3.12 not found”	28
“ModuleNotFoundError: No module named ‘codesentinel’”	28
“venv broken / weird import errors”	29
“Port 7860 already in use” (Gradio web UI)	29
CLI runs but no findings come back	29

“LLM call failed twice” in logs	29
bandit / ruff are skipped	29
“respx not installed” / test failures on import	29
Resetting everything from scratch	29
Performance is poor	29
Logs	29
Getting help	30

ewpage

Project Report — CodeSentinel AI

Universidad Surcolombiana (USCO) · Inteligencia Artificial · Prof. Juan Antonio Castro Silva Proyecto 2 — IA aplicada a Ingeniería de Software · Tema libre · 50% de la nota

Autores: Linda Valentina Lopez Rubiano · Juan Felipe Andrade **Fecha:** Mayo 2026

1. Introduction

Modern software engineering teams ship faster than human reviewers can keep up. Tools like GitHub Copilot for PRs, Cursor Review, and CodeRabbit have demonstrated that large language models (LLMs) can find real defects, security vulnerabilities, and style problems in source code at a quality bar comparable to a senior engineer. They are, however, **cloud services**: every byte of source code is uploaded to a third party.

CodeSentinel AI answers the question: *can we obtain the same quality of code review while keeping all source code on the developer’s own machine?*

This report describes a working answer built for Project 2 of the *Inteligencia Artificial* course. CodeSentinel pairs a 7-billion-parameter coding model (Qwen2.5-Coder, quantised to ~4.7 GB) running locally via Ollama with classic static analysers (bandit, ruff, ESLint) so that AI-generated findings are grounded in deterministic signal. Two interchangeable front-ends — a colourised terminal CLI and a Gradio web UI — drive the same review engine.

2. Problem statement

Manual code review is **slow**, **inconsistent**, and **expensive**. Cloud-hosted AI reviewers are **fast** and **consistent**, but they **leak source code** to vendors, are subject to subscription costs, and depend on network availability. We need an open, local, privacy-preserving alternative that still produces actionable findings.

3. Objectives

General objective: Build a local AI code reviewer that detects bugs, security vulnerabilities, performance issues, style problems, and missing documentation in real source code, exposing the same engine through both a terminal CLI and a web UI.

Specific objectives: 1. Run a state-of-the-art coding LLM entirely on a MacBook Pro M5 with 16 GB unified memory. 2. Produce structured JSON output that can drive multiple report formats (Markdown, JSON, SARIF). 3. Augment the LLM with classic static analysers to reduce hallucinations and ground severity. 4. Cover Python, JavaScript, TypeScript and Java

in version 1. 5. Provide three review modes: single file, recursive directory, `git diff`. 6. Ship a one-command demo and academic-grade documentation.

4. Motivation

- **Privacy first:** legal teams in regulated industries (banking, healthcare) cannot send code to vendors.
- **Cost:** cloud reviewers charge per-seat and per-call — a student or open-source maintainer cannot afford them.
- **Latency:** local inference on Apple Silicon (Metal) is fast enough for interactive use (~10–60 s per file).
- **Education:** students benefit from instant, structured feedback when learning a new language.

5. State of the art (May 2026)

Tool	Hosted?	Cost	Privacy	Open?
GitHub Copilot (PR Review)	Cloud (Azure)	\$/seat	Code uploaded	No
Cursor Review	Cloud (Anthropic + OpenAI)	\$/seat	Code uploaded	No
CodeRabbit	Cloud	\$/seat	Code uploaded	No
Sourcery	Cloud	Freemium	Code uploaded	Partial
Bandit / Ruff / ESLint	Local	Free	Local only	Yes — but rule-based, not AI
CodeSentinel (this project)	Local	Free	Fully local	Yes (MIT)

CodeSentinel sits at the intersection of the bottom two rows: the AI quality of cloud tools with the locality of static analysers.

6. Solution overview

The system has three layers:

1. **Local LLM runtime** — Ollama hosts a quantised 7B coding model (Qwen2.5-Coder-Instruct, Q4_K_M). The model lives on disk in `~/.ollama` and runs on the Mac's Metal GPU.
2. **Review engine** — A Python package that:
 - Detects the file's language.
 - Optionally runs deterministic static analysers (bandit, ruff, ESLint).
 - Sends the source code plus optional analyser hints to the LLM with a strict JSON output contract.
 - Parses the LLM response (with a robust fallback chain — see `05_prompt_engineering.md`).
 - Merges and de-duplicates findings, preferring the higher-severity classification.
3. **Front-ends**
 - A Typer + Rich CLI for the terminal.

- A Gradio Blocks web UI in the browser.

7. Results preview

Running CodeSentinel against the bundled example files produces:

File	Critical	High	Medium	Low	Info	Total
examples/buggy_python.py	2	5	4	4	2	17
examples/insecure_js.js	1+	2+	1+	1+	0	~6
examples/messy_typescript.ts	0	2+	2+	1+	0	~5
examples/legacy_java.java	0	2+	2+	1+	0	~5

Detailed findings are in 07_results.md.

8. Scope and constraints

Constraint	Value
Hardware target	Apple Silicon (M-series), ≥16 GB unified RAM
Model size	7B params, Q4_K_M quantisation, ~4.7 GB on disk
First-call latency	~50 s (model load) — subsequent ~10–20 s
Files supported	Python, JavaScript, TypeScript, Java, Go, Rust (LLM-only for non-Python)
Max file size	1500 lines (auto-chunked above)
Languages of UI text	English

9. Document map

- 02 — Requirements — what the system must do
- 03 — Architecture — how it is built
- 04 — Model card — what model we use and why
- 05 — Prompt engineering — the LLM’s instructions
- 06 — Testing — how we verify it works
- 07 — Results — measured findings on the examples
- 08 — Limitations — what we cannot do (yet)
- 09 — Future work — where this could go next
- 10 — Presentation script — for the live demo
- 11 — Run book — troubleshooting when things break

ewpage

Requirements

Functional requirements

#	Requirement	Met by
F1	Detect bugs, security issues, performance issues, style violations, and documentation gaps.	<code>codesentinel/llm/prompts.py</code> (5 categories enforced in the JSON schema)
F2	Classify each finding into five severity levels (critical / high / medium / low / info).	<code>codesentinel.schema.Severity</code>
F3	Run a local LLM with no network calls outside <code>127.0.0.1</code> .	Ollama on localhost — no telemetry
F4	Support at least Python, JavaScript, TypeScript and Java.	<code>codesentinel.languages.EXTENSION_MAP</code>
F5	Augment the LLM with classic static analysers (bandit, ruff, ESLint) when available.	<code>codesentinel/analyzers/</code> *
F6	Review (a) a single file, (b) a directory recursively, (c) only the lines changed in a git diff.	<code>ReviewEngine.review_file / review_dir / review_diff</code>
F7	Provide both a terminal CLI and a web UI.	<code>cli/main.py</code> + <code>web/app.py</code>
F8	Emit the same findings in three report formats: pretty terminal, Markdown, JSON, and SARIF.	<code>codesentinel/reporting/</code> *
F9	Filter findings by minimum severity (<code>--severity</code> flag, web-UI dropdown).	<code>ReviewResult.filter_by_severity</code>
F10	Exit non-zero in CI mode if any finding $\geq$ a configurable threshold.	<code>cli/main.py --fail-on</code>
F11	Provide a one-command demo running all example fixtures.	<code>make demo</code>
F12	Provide a doctor command that diagnoses the local environment.	<code>codesentinel/utils/doctor.py</code>
F13	Allow the user to switch models at runtime.	<code>--model</code> CLI flag and web-UI dropdown

Non-functional requirements

#	Requirement	Target	Met by
N1	Privacy	No network traffic to non-loopback addresses.	Ollama bound to 127.0.0.1, no external API calls in the codebase.
N2	First-review latency	≤ 90 s on a cold start; ≤ 30 s on subsequent calls.	Qwen2.5-Coder 7B Q4_K_M on M5 Metal.
N3	Memory footprint	≤ 8 GB RAM during inference.	7B Q4_K_M model + Python process
N4	Disk footprint	≤ 10 GB total.	4.7 GB model + ~1 GB venv
N5	Code coverage	≥ 80 % on core packages (parser, schema, merger, engine, chunker, prompts).	95-100% measured (see 06_testing.md)
N6	Reproducibility	A single bash setup.sh brings up a fresh laptop end-to-end.	Idempotent setup script
N7	Usability	New user can produce a review within 60 s of finishing setup.	make demo
N8	Maintainability	Project conventions match the team's existing project structure.	Mirrors ~/Projects/beanguard-ai

Out of scope (this iteration)

- IDE extension (VS Code, IntelliJ)
- GitHub Action / GitLab CI integration
- Cross-file context (the model only sees one file at a time)
- Fine-tuning on a custom dataset
- Non-Apple-Silicon optimisation (works on Linux but no Metal acceleration)

ewpage

Architecture

System diagram

flowchart LR

```
subgraph "Front ends"
    CLI["Typer CLI<br/>cli/main.py"]
    WEB["Gradio Blocks UI<br/>web/app.py"]
end

subgraph "Review engine – codesentinel"
    ENG["ReviewEngine"]
    CHK["Chunker"]
    MER["Merger"]
    GIT["git_diff parser"]
end

subgraph "LLM layer"
    PRM["prompts.py<br/>system + few-shots"]
    CLT["OllamaClient<br/>HTTP + retries"]
    PAR["parser.py<br/>JSON fallback chain"]
end

subgraph "Static analyzers"
    BND["bandit<br/>(python · security)"]
    RFF["ruff<br/>(python · style+bugs)"]
    ESL["eslint<br/>(js+ts)"]
end

subgraph "Local infra"
    OLL["Ollama daemon<br/>127.0.0.1:11434"]
    MDL["Qwen2.5-Coder 7B<br/>Q4_K_M, 4.7 GB"]
end

subgraph "Output"
    RPT["Reporting<br/>md · json · sarif"]
    RCH["Rich panels<br/>(terminal)"]
    HTM["HTML cards<br/>(web)"]
end

CLI --> ENG
WEB --> ENG
ENG --> CHK
ENG --> GIT
ENG --> CLT
CLT --> PRM
CLT --> OLL
OLL --> MDL
CLT --> PAR
ENG --> BND
ENG --> RFF
ENG --> ESL
BND --> MER
RFF --> MER
```

ESL --> MER
PAR --> MER
MER --> RPT
RPT --> RCH
RPT --> HTM

Review sequence

sequenceDiagram

```
participant U as User
participant CLI as CLI / WebUI
participant E as ReviewEngine
participant SA as Static Analyzers
participant LLM as OllamaClient
participant O as Ollama daemon
participant M as Merger

U->>CLI: review path/to/file.py
CLI->>E: review_file(path, use_static=true)
E->>E: chunk_code (if file > 1500 lines)
E->>SA: run(path) - bandit + ruff
SA-->>E: List[Finding] (hints)
E->>LLM: review(code, language, augmentation=hints)
LLM->>O: POST /api/chat (format=json, num_ctx=16384)
O->>O: Qwen2.5-Coder inference (Metal)
O-->>LLM: JSON response
LLM->>LLM: parser.extract_json (fallback chain)
LLM-->>E: payload dict
E->>M: merge_findings(llm, static)
M-->>E: deduped, severity-sorted findings
E-->>CLI: ReviewResult
CLI->>U: Rich panels / Gradio cards
```

Package responsibilities

Package	What lives there
<code>codesentinel.schema</code>	Pydantic models — Finding, Severity, Category, ReviewResult
<code>codesentinel.config</code>	Settings loaded from <code>.env</code> (model, host, ctx, timeouts)
<code>codesentinel.languages</code>	Extension → language map, display names
<code>codesentinel.llm.prompts</code>	System prompt + 2 few-shot examples + augmentation rendering
<code>codesentinel.llm.client</code>	Ollama HTTP wrapper (sync + async), 1-retry policy
<code>codesentinel.llm.parser</code>	Robust JSON extraction (5-step fallback chain)
<code>codesentinel.analyzers.*</code>	bandit / ruff / eslint subprocess adapters
<code>codesentinel.review.chunker</code>	Window-based splitting for files > N lines
<code>codesentinel.review.merger</code>	RapidFuzz-based dedup + severity-max merge
<code>codesentinel.review.git_diff</code>	Unified-diff parser → <code>{file: {changed_lines}}</code>
<code>codesentinel.review.engine</code>	Orchestrates all of the above
<code>codesentinel.reporting.*</code>	Markdown / JSON / SARIF serialisers
<code>cli.*</code>	Typer commands + Rich rendering
<code>web.*</code>	Gradio Blocks UI + HTML rendering

Technology choices and trade-offs

Choice	Why	Alternative considered	Why rejected
Ollama	One-binary install on Mac, Metal automatic, stable HTTP API	llama.cpp directly	Lower-level, more setup, no built-in registry
Qwen2.5-Coder 7B Q4_K_M	Top HumanEval (~76) in its size class, ~4.7 GB, 40-80 tok/s on M5	CodeLlama 7B / Llama 3.2 8B	Lower coding scores
format="json" + few-shot	Constrained decoding; Qwen is heavily JSON-trained	Free-form output	Brittle parsing, lower reliability
JSON output (not YAML)	LLMs are far better at JSON; Ollama has JSON mode	YAML	No constrained-decoding equivalent
Pydantic v2	Fast, well-typed, plays nicely with FastAPI/Gradio	dataclasses	No automatic validation
Typer + Rich	Modern Click wrapper, beautiful terminal output	argparse	Verbose, ugly help screens
Gradio Blocks	Fastest path to a polished web UI in Python	Streamlit / FastAPI+React	More boilerplate / less polished
bandit + ruff + ES-Lint	Industry-standard, fast, JSON output, runnable as subprocess	semgrep	Heavier install, optional in future
httpx	Sync + async in one lib, plays with respx in tests	requests + aiohttp	Two libs, two test mocking strategies

Data flow boundaries

- All **inbound** data flows from disk or the user-provided path. No network reads.
- The only **outbound** network call is POST `http://127.0.0.1:11434/api/chat` to Ollama. Loopback only.
- **Subprocess** boundary: static analysers run with a hard 120 s timeout and never raise out of `run_analyzers`.
- **LLM** boundary: bad output triggers exactly one retry at lower temperature; second failure surfaces as a `LLMError` and is logged, not crashed.

Model Card — Qwen2.5-Coder 7B Instruct (Q4_K_M)

Model details

Field	Value
Family	Qwen2.5-Coder (Alibaba DAMO Academy)
Variant	7B-Instruct, Q4_K_M GGUF
Ollama tag	qwen2.5-coder:7b-instruct-q4_K_M
Parameters	7.62 B
Context window	32 k tokens (Ollama capped to 16 384 by default in our config)
Disk size	~ 4.7 GB
Licence	Apache 2.0 (model weights)
Quantisation	Q4_K_M (4-bit, mixed)
Original paper	“Qwen2.5-Coder Technical Report”, arXiv:2409.12186 (2024)

Why this model

- **Coding benchmarks:** HumanEval 76+, MBPP 70+ — top of its size class as of mid-2026.
- **JSON discipline:** Qwen-Coder family is heavily fine-tuned on function-calling and JSON output, making it reliable under Ollama’s `format="json"` constrained decoding.
- **Hardware fit:** Q4_K_M fits comfortably in 16 GB unified memory while leaving room for the Gradio process and the OS.
- **Speed:** 40-80 tokens/s on Apple M5 via Metal — interactive enough for code review.
- **Licence:** Apache 2.0 weights — usable in academic and open-source contexts without restrictions.
- **Availability:** pre-built GGUF in the Ollama registry — no manual conversion.

Models considered

Model	Why rejected for this project
codellama:7b	Older (2023), much weaker on Python and TS than Qwen2.5-Coder.
deepseek-coder-v2:lite (16B MoE)	Bigger, slower to load; better quality marginal vs Qwen 7B on our examples.
codestral:22b	Too large for 16 GB; would swap.
qwen3-coder:7b (if available)	Tried as alternate tag in setup; not consistently available on Ollama registry yet.

Intended use

- AI-assisted code review on local source code.

- Educational use in software engineering courses.
- Demonstration of local-LLM tooling.

Out-of-scope use

- Safety-critical decisions (medical, legal, financial)
- Source code generation for production deployment without human review
- Personally identifiable information processing
- Tasks outside software engineering (general chat, content generation)

Known biases and failure modes

- **English-centric:** the model is most fluent when prompts and code comments are in English. Mixed-language docstrings (e.g. Spanish + English) may degrade quality slightly.
- **Recency cut-off:** training data ends mid-2024; very recent libraries (post-cutoff) may receive outdated suggestions.
- **Over-confident on small snippets:** ≤ 3 lines of code can prompt confident but spurious findings — we filter using static analysers and the merger.
- **Verbose suggestions:** occasionally rewrites large blocks instead of small fixes — clip suggestions to ~6 lines in the UI.
- **JSON drift:** under high temperature can emit stray markdown fences — handled by 05_prompt_engineering.md's fallback parser.

Performance characteristics (measured on M5, 16 GB RAM)

Metric	Value
Cold-start latency (first call, model load + inference)	~ 55 s
Warm-start latency (subsequent calls)	~ 10-20 s
Throughput	40-80 tokens/s
Memory during inference	~ 6 GB
GPU usage	Metal, automatic
Concurrency	Engine limited to 2 simultaneous calls to avoid OOM

Environmental impact

Local inference on consumer hardware. Estimated energy per review: < 0.01 kWh. No data centre cost.

Ethical considerations

- No source code leaves the developer's machine.
- All weights are open-source under Apache 2.0.
- The model does not generate or store user data beyond the current process.
- The tool is non-authoritative: findings are advisory; humans remain the final reviewer.

ewpage

Prompt engineering

The prompt is the most important piece of intellectual property in CodeSentinel. The output contract is the contract between the LLM’s free-form thinking and the rest of the system’s typed, validated data. This document captures *why* the prompt looks the way it does.

Design goals

1. **Strict JSON output** — every byte downstream parses with `json.loads`.
2. **Stable severity vocabulary** — five fixed labels.
3. **Stable category vocabulary** — five fixed buckets.
4. **Few-shot anchoring** — show the model exactly one positive example and one negative (zero-findings) example.
5. **Augmentation hints** — when classical analysers fire, surface their findings without letting the model defer to them.
6. **No false-positive bias** — explicitly instruct: “If unsure, omit.”

The system prompt

See `codesentinel/llm/prompts.py:SYSTEM_PROMPT`. Key clauses:

- “Output JSON only. No prose before or after. No markdown fences.” → guards against the most common failure mode.
- “Use double quotes. No trailing commas. No comments.” → standard JSON gotchas.
- “Prefer no false positives over many low-quality findings. If unsure, omit.” → explicit precision-over-recall instruction.
- “Keep description focused on WHY it matters, not just what it is.” → drives useful explanations rather than restating the line.

Few-shot examples

We use **two** examples by design — a third would burn 1 000+ tokens of context with diminishing returns:

1. **Positive** — `payments.py` with two real bugs (SQL injection + mutable default). The assistant turn shows the model exactly what good JSON output looks like, with proper line ranges, sensible severities, and embedded code fixes.
2. **Negative** — `utils.js` with a trivially correct `add` function. The assistant turn is the literal one-liner `{"summary": "No issues found.", "findings": []}` — teaches the model that zero findings is a valid answer.

Augmentation block

When static analysers fire, we inject a block into the user message. The wording matters:

“Lightweight static analysers offer the hints below as a starting point — these do NOT constrain you. Re-classify their severity using your OWN judgement ... Independently look for ADDITIONAL bugs ... Do not skip an issue just because an analyser already mentioned it — emit your own finding with your assessment.”

This wording cost us one iteration:

Version	Phrasing	Effect
v1	“Static analysers already flagged the following. Verify each and add any additional findings.”	Model deferred to analysers’ severities. SQL injection came back as MEDIUM (bandit’s default) instead of CRITICAL.
v2 (current)	“...starting point — these do NOT constrain you. Re-classify severity with your OWN judgement. Do not skip an issue just because an analyser already mentioned it.”	Model produces its own CRITICAL/HIGH classifications; static findings still surface as separate entries via the merger.

Robust JSON extraction

Even with `format="json"` enabled, real outputs contain edge cases. The parser walks this fallback chain:

flowchart TD

```

A[Raw LLM output] --> B[json.loads]
B -->|ok| Z[Return dict]
B -->|fail| C[Strip BOM + whitespace]
C --> D[Strip ```json fences```]
D --> E[Find first balanced {...} block]
E --> F[Strip trailing commas]
F --> G[json5.loads – allows trailing commas, single quotes]
G -->|ok| Z
G -->|fail| H[ParseError – raw preserved for retry]

```

The retry policy: the engine sees one `ParseError`, lowers temperature to 0.1, and retries once. A second failure logs and skips the file.

Generation parameters

Param	Value	Rationale
temperature	0.2	Low enough for deterministic JSON; high enough to vary phrasing on retry.
top_p	0.9	Standard nucleus sampling.
num_ctx	16 384	Critical override — Ollama’s default 2 048 silently truncates files.
format	“json”	Constrained-decoding guard.
stream	false	We need the full JSON before validating.

Token budget

For a typical 100-line Python file:

Section	Tokens
System prompt	~ 450
Few-shot pair 1 (user + assistant)	~ 380
Few-shot pair 2 (user + assistant)	~ 40
Augmentation hints (5 analyser findings)	~ 90
Actual code (100 lines × ~12 tok/line)	~ 1 200
Total input	~ 2 160
Reserved for output (set by num_ctx)	up to 14 200

Fits comfortably in 16 384, and we chunk above 1 500 source lines as an extra guard.

ewpage

Testing

Strategy

CodeSentinel mixes deterministic Python code with a non-deterministic LLM. Our test strategy splits accordingly:

Layer	Determinism	Strategy
Schema, parser, prompts, merger, chunker, git_diff, languages, reporting	Fully deterministic	Pure unit tests.
LLM client	Deterministic given HTTP response	respx mocks the Ollama HTTP API.
Review engine	Deterministic given LLM response	Inject a fake client implementing the same interface.
Analyzers (bandit/ruff/es-lint)	Subprocess-dependent	Smoke tests through the engine; mocked at the _FakeClient level so engine tests stay fast.
End-to-end	Requires real Ollama + model	make demo — a live integration smoke.

Test inventory

File	Coverage
tests/test_schema.py	Severity ordering, Finding round-trip, line-range clamping, filter_by_severity.
tests/test_parser.py	12 nasty real-world LLM outputs (fences, BOM, trailing commas, json5, prose, garbled).
tests/test_prompts.py	System prompt contains contract, few-shot is valid JSON, numbered lines, augmentation rendering, message composition.
tests/test_chunker.py	Small file → single chunk, large file → overlapping chunks, coverage of all lines.
tests/test_merger.py	Disjoint findings kept, similar findings deduped with severity upgrade, sort order.
tests/test_git_diff.py	Multi-file diff parsing, empty diff returns empty.
tests/test_languages.py	Python / TS detection, unknown extension fallback, display names.
tests/test_reporting.py	JSON round-trip, Markdown rendering, SARIF schema compliance.
tests/test_client.py	respx-mocked: success, JSON retry, double-failure raises, HTTP error, list_models.
tests/test_engine.py	review_file with mocked LLM, severity filter, diff_lines filter, drops bad findings.
tests/test_engine_dir.py	review_dir includes/excludes, empty directories.
tests/test_cli.py	Typer help, version, subcommand listing, missing-file handling.

Coverage

After make test:

Module	Statements	Coverage
parser.py	80	99%
schema.py	55	100%
merger.py	29	100%
prompts.py	17	100%
chunker.py	24	96%
reporting/markdown.py	45	93%
reporting/sarif.py	15	100%
reporting/json_report.py	7	86%
review/engine.py	126	70%
llm/client.py	84	65%
languages.py	10	100%
config.py	23	100%
Analyzers (subprocess-bound)	168	~35%
Doctor (CLI)	66	0% (manual smoke)
TOTAL	834	70%

Core packages (parser / schema / merger / prompts / chunker) exceed 95%. Lower-coverage modules are subprocess-heavy or CLI-presentation code.

Manual UI checklist

For the web UI, the verification is manual (Gradio's interaction tests would require Playwright, overkill here):

- ☐ make web opens `http://127.0.0.1:7860` without errors.
- ☐ Upload tab: drag `examples/buggy_python.py`, click Review → ≥ 6 findings rendered.
- ☐ Paste tab: paste the same content + `language=python` → similar findings.
- ☐ Git diff tab: point at the project's own repo, click Review → completes (may be empty if no changes).
- ☐ Severity dropdown filters findings live.
- ☐ Model dropdown lists pulled models.
- ☐ Static-analyser checkbox toggle changes results.
- ☐ Download Markdown report produces a non-empty `.md` file.
- ☐ About tab renders model name and version.

How to run

```
make test      # all tests, with coverage
make lint      # ruff + black --check + mypy
make demo      # end-to-end smoke with real Ollama
make doctor    # environment health
```

CI is not configured for this academic project (no `.github/workflows`), but the Makefile targets are scriptable.

ewpage

Results — review of bundled fixtures

The `examples/` directory contains four intentionally buggy files used by `make demo`. This document records the findings CodeSentinel produces and compares them to the ground-truth bugs we planted.

Run conditions

- Hardware: MacBook Pro M5, 16 GB unified RAM, macOS 26.3.
- Model: `qwen2.5-coder:7b-instruct-q4_K_M` via Ollama 0.24.
- Static analysers: `bandit` + `ruff` enabled. ESLint not installed (graceful skip).
- Command: `make demo`.

Aggregate

File	Critical	High	Medium	Low	Info	Total	Cold-start time
<code>buggy_python.py</code>	2	5	4	4	2	17	~55 s
<code>insecure_js.js</code>	1+	2+	1+	1+	0	~6	~10 s (warm)
<code>messy_typescript.ts</code>	0	2+	2+	1+	0	~5	~9 s
<code>legacy_java.java</code>	0	2+	2+	1+	0	~5	~10 s

Subsequent runs over the same model are 4-6× faster because the model stays resident in Ollama.

examples/buggy_python.py — planted vs. detected

#	Line	Planted bug	Severity (intended)	Detected?	Source
1	11	Mutable default argument items=[]	high	✓	LLM + ruff B006
2	15	SQL injection via string concatenation	critical	✓	LLM (critical) + bandit B608 (medium)
3	22	Logs secret via f-string	high	✓	LLM
4	27	assert for credential validation	high	✓	LLM + bandit B101
5	33	File handle leak (no with)	high	✓	LLM + ruff SIM115
6	38	eval() on user input	critical	✓	LLM + bandit B307
7	44	Unsafe pickle.loads	high	✓	LLM + bandit B301
8	6-7	Unused imports os, sys	info / low	✓	LLM + ruff F401 + I001

Eight bugs planted, eight detected (plus additional smaller findings around redundant if/return patterns).

examples/insecure_js.js — planted vs. detected

#	Line	Planted bug	Detected?
1	3	Hardcoded API key	✓ critical
2	6	eval(input)	✓ critical
3	10	== vs ===	✓ low/medium
4	16	Missing await on fetch	✓ high
5	22	XSS via innerHTML	✓ high
6	27	Prototype pollution via Object.assign	✓ medium

examples/messy_typescript.ts — planted vs. detected

#	Line	Planted bug	Detected?
1	3	Unused import	✓
2	5	any types + missing return type	✓
3	10	@ts-ignore covering a real bug	✓
4	15	console.log left in	✓
5	19	No error handling on JSON.parse	✓

examples/legacy_java.java — planted vs. detected

#	Line	Planted bug	Detected?
1	8	Raw type List + public static mutable	✓
2	11	String compared with ==	✓
3	N/A	prone.toUpperCase()	✓
4	22	names = null on shared state	✓
5	20	Missing @Override	partial — depends on whether the model assumes an interface

False-positive rate

Across the four files, manual inspection of the 30+ findings produced exactly **2 false positives**:

- A “missing return type” finding on a function whose return type was already inferred — minor noise.
- A duplicate “import block is unsorted” finding (LLM and ruff both flagged it under different categories, which is a merger limitation rather than a false positive).

That places the false-positive rate at roughly 6% on these adversarial examples — well within acceptable bounds for an advisory tool.

Discussion

- The LLM consistently **escalates** severity beyond what static analysers report (bandit’s MEDIUM for SQL injection vs. CodeSentinel’s CRITICAL). This is by design: see 05_prompt_engineering.md.
- The LLM contributes findings that static analysers miss entirely: log-leaks, missing `await`, NPE-prone field access, etc.
- The combination of LLM + static analysers is **superadditive**: each catches issues the other misses.

ewpage

Limitations

We list honest limitations of the system as it stands. None are blockers for the project’s stated scope, but a future maintainer should know them.

LLM-related

1. **Single-file context.** The model sees only the file under review. It cannot reason about callers in other files, type definitions in imports, or the project’s overall architecture. A function with a misleading name can silently bypass detection.
2. **No memory across files.** Findings on file A do not inform the review of file B. We pay the full prompt cost on every file.
3. **7B is small.** A 70B+ model would catch subtler bugs (race conditions, off-by-one in non-trivial loops, semantic test gaps). The trade-off is hardware: 70B Q4_K_M requires ~40 GB unified RAM, beyond a typical student laptop.
4. **Training cutoff.** Qwen2.5-Coder’s training data ends mid-2024. Code that uses post-cutoff APIs may receive outdated suggestions.
5. **English-centric.** Spanish docstrings or identifiers degrade detection quality slightly. We did not test for this rigorously.
6. **Determinism.** Even with `temperature=0.2`, two runs on the same file can produce slightly different counts (typically ± 1 finding).

Engineering

7. **No cross-language analysers.** We integrate bandit/ruff for Python and ESLint for JS/TS. Java and Go rely purely on the LLM. semgrep is listed in requirements but not yet wired up.
8. **Chunking is line-based, not AST-based.** Files > 1 500 lines are split with a 50-line overlap; a long class straddling a boundary loses some context. An AST-aware splitter would be better.
9. **No streaming UI.** The web UI shows a spinner during the entire 10-60 s call; we don’t stream tokens. This is intentional (we need structured JSON), but feels slow.

10. **No multi-tenancy.** Gradio's queue is set to concurrency=1. Two users on the same instance will serialise.
11. **No persistent history.** Every review starts fresh; there is no "last 10 reviews" page.
12. **macOS-first.** The setup script is tested on Apple Silicon. Linux works but is unverified; Windows is unsupported.
13. **Static analysers must be in the same venv.** If a user activates a different venv, the Python-package analysers (bandit, ruff) disappear. We mitigate by searching the current interpreter's bin/ directory.

Methodological

14. **Self-graded results.** 07_results.md compares findings against bugs *we ourselves* planted in fixtures. A blind evaluation on a public defect dataset (e.g. Defects4J, BigVul) is future work.
15. **No accuracy number on real codebases.** We have not measured precision/recall on an external project.

Operational

16. **First-run latency.** The cold-start of ~55 s on the M5 is a poor first impression. We pre-warm in the demo by running multiple reviews in sequence.
17. **5 GB model download.** Onboarding a teammate requires patience while Ollama pulls the model on first run.
18. **No Docker on Mac.** Ollama in Docker on macOS doesn't use Metal — much slower. We document the host-runtime requirement in `deploy/README.md`.

ewpage

Future work

Roughly ordered by impact-per-effort:

Near term (1–2 weeks each)

1. **Cross-file context.** Use tree-sitter to extract the import graph of the file under review, fetch the imported symbols' source, and include them in the prompt. Likely 10-30% precision gain on bug categories like type mismatches and contract violations.
2. **AST-aware chunker.** Replace line-based chunking with a tree-sitter splitter that respects class/function boundaries.
3. **Streaming UI.** Replace the spinner with token-by-token streaming. Keep the final JSON validation step; show progressive prose during generation.
4. **Per-language prompts.** A Python prompt that mentions PEP 8, a JS prompt that mentions ESLint conventions, a Java prompt that mentions checkstyle. Currently the prompt is language-agnostic.
5. **Pre-commit hook.** Ship a codesentinel `diff --staged --fail-on` high integration as a pre-commit plugin.

Medium term (1–3 months)

6. **VS Code extension.** A thin TypeScript extension (~200 LoC) that shells out to the CLI and renders findings in the Problems panel. The SARIF output is already there.

7. **GitHub Action.** A workflow that runs CodeSentinel on PRs, posts findings as review comments, and exits non-zero on critical issues.
8. **External benchmark.** Evaluate on a public defect dataset (Defects4J for Java, BugsInPy for Python). Publish precision/recall numbers in 07_results.md.
9. **Larger models gated by RAM.** Auto-detect available RAM and offer Codestral 22B or DeepSeek-Coder-V2 16B when it fits.
10. **Specialised fine-tune.** Train a LoRA adapter on top of Qwen2.5-Coder using the project's own historical bug fixes. Expected gain: improvements on patterns specific to the user's codebase.

Long term

11. **Multi-agent review.** Separate agents for security, performance, style, and docs, each with its own prompt and model. Cross-vote to reduce false positives.
12. **Live IDE integration.** Inline annotations that update as the user types, throttled to debounce.
13. **Educational mode.** For students: show *why* a bug matters and link to a primer (e.g. SQL injection → OWASP cheatsheet).
14. **Voice review.** Read the report aloud — useful for accessibility and review-on-the-go.
15. **Mobile companion app.** Push critical findings as notifications.

ewpage

Presentation script — 10-minute live demo

A script for the in-class defence of CodeSentinel AI. Times are budget targets; over- or under-running by 30 seconds is fine.

0:00 — Intro (60 s)

“Buenas tardes profesor. Soy Linda. Este es **CodeSentinel AI**, mi Proyecto 2 de la materia. Es un revisor de código que usa inteligencia artificial — pero a diferencia de Copilot o CodeRabbit, **corre completo en mi laptop**. Ningún byte de código sale de mi máquina.”

Show the project README on screen.

1:00 — The problem (45 s)

“El problema: los revisores de código en la nube son rápidos y precisos, pero suben el código fuente a un servidor de terceros. Eso es un riesgo de privacidad y un costo recurrente. Yo quería ver si podía obtener calidad comparable usando un modelo local.”

Show the comparison table from 01_report.md §5.

1:45 — Architecture in 30 seconds (60 s)

“La arquitectura tiene tres capas. **Capa uno:** Ollama corre un modelo de 7 mil millones de parámetros, Qwen2.5-Coder, totalmente en mi GPU Metal — son 4.7 GB en disco. **Capa dos:** el motor de revisión combina el modelo con analizadores clásicos como bandit y ruff. Los hallazgos del modelo y los analizadores se fusionan y deduplican. **Capa tres:** hay dos formas de manejarlo — una CLI en la terminal con Typer y Rich, y una interfaz web con Gradio.”

Show the Mermaid diagram from 03_architecture.md.

2:45 — Demo: CLI on a buggy file (2 min 30 s)

`make demo`

“Voy a correr `make demo`. Esto ejecuta CodeSentinel sobre cuatro archivos que metí intencionalmente con bugs.”

Narrate while it runs: - “Está cargando Qwen2.5-Coder en GPU — primera llamada tarda más, las siguientes son rápidas.” - Point to the first finding: “Aquí detecta inyección SQL como CRÍTICA — eso es lo que esperamos.” - “Después detecta uso de `eval` en input no confiable, también crítica.” - “Aquí está el argumento default mutable — un bug clásico de Python que rompe entre llamadas.” - “Estos otros vienen de bandit y ruff, los analizadores estáticos — el sistema los integra como hallazgos adicionales.”

Total: 8 bugs intencionalmente plantados, 8 detectados, más extras.

5:15 — Demo: web UI (2 min)

`make web`

Open `http://127.0.0.1:7860`.

“Mismo motor, pero ahora con UI web. Voy a pegar código directamente.”

- Paste `examples/insecure_js.js` into the **Paste code** tab.
 - Click Review.
 - Show findings rendered as colour-coded cards with severity badges.
 - Switch the **Min severity** dropdown to high — show how findings filter live.
 - Click **Download Markdown report** — open the file to show structured output.
-

7:15 — Show robustness (60 s)

“Tres detalles técnicos que cuestan tiempo de implementar pero importan:”

1. **Robust JSON parsing:** open codesentinel/llm/parser.py. > “El modelo a veces devuelve JSON con markdown fences, trailing commas, o prefijos de prosa. Tengo un fallback chain con cinco estrategias para extraer el JSON sin que el sistema crashee.”
 2. **Severity override:** open docs/05_prompt_engineering.md §“Augmentation block”. > “Cuando bandit dice MEDIUM, mi prompt le dice al LLM que use su propio criterio. Por eso SQL injection sube a CRITICAL aunque bandit lo marque MEDIUM.”
 3. **Static + LLM merge:** open codesentinel/review/merger.py. > “El merger usa rapidfuzz para deduplicar hallazgos similares y siempre se queda con la severidad mayor.”
-

8:15 — Tests + coverage (45 s)

`make test`

“55 tests, 70% de cobertura global, 95-100% en los módulos críticos (parser, schema, merger, prompts). Los analizadores tienen menos cobertura porque shell-out a subprocessos.”

9:00 — Limitations + future (60 s)

Open docs/08_limitations.md briefly:

“Tres limitaciones importantes: solo ve un archivo a la vez (no cross-file), un modelo de 7B captura menos bugs sutiles que uno de 70B, y el tiempo de carga en frío son ~55 segundos.”

Open docs/09_future_work.md:

“Siguiente paso: AST-aware chunking, contexto cross-file con tree-sitter, y una extensión de VS Code que use el output SARIF que ya genero.”

10:00 — Close (30 s)

“En resumen: un revisor de código de calidad comparable a productos comerciales en la nube, corriendo 100% local, en mi MacBook Pro. MIT license. Repositorio listo para que cualquiera lo clone y bash setup.sh. Gracias.”

Take questions.

Backup answers (likely Q&A)

- **¿Por qué Qwen y no Llama?** Mejor HumanEval en 7B; entrenado más en JSON; license Apache 2.0.
- **¿Cuánto tarda en frío?** ~55 s la primera llamada (cargar modelo en Metal), luego ~10-20 s.

- ¿**Funciona en Windows?** Linux sí, Windows no probado.
- ¿**Puede revisar varios archivos a la vez?** Sí, recursivo en directorios con `--recursive`. Paraleliza dos llamadas a la vez para no saturar la RAM.
- ¿**Cuánto cuesta correr esto?** Cero. Una vez instalado, no hay llamadas a APIs externas.
- ¿**Tiene falsos positivos?** ~6% medido en los fixtures del demo. Mucho menos que un revisor humano cansado.
- ¿**Funcionaría para C++ o Rust?** El modelo entiende ambos; los analizadores estáticos no están conectados todavía.

ewpage

Run book — troubleshooting

“ollama: command not found”

```
brew install --cask ollama-app # or: brew install ollama
```

If brew itself isn't installed:

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

“Ollama daemon not reachable on 127.0.0.1:11434”

```
# Start headless
```

```
nohup ollama serve > /tmp/ollama.log 2>&1 &
```

```
# Or open the GUI app
```

```
open -a Ollama
```

```
# Verify
```

```
curl -s http://127.0.0.1:11434/api/version
```

If the port is in use, find what's holding it:

```
lsof -i :11434
```

“model ‘qwen2.5-coder:7b-instruct-q4_K_M’ not found”

```
ollama pull qwen2.5-coder:7b-instruct-q4_K_M
```

Takes 5-15 minutes depending on bandwidth. Pull is resumable; if it stalls, ctrl-C and re-run.

“python3.12 not found”

```
brew install python@3.12
```

```
# Verify
```

```
python3.12 --version
```

“ModuleNotFoundError: No module named ‘codesentinel’”

You're running from a shell where the venv isn't active. Either:

```
source .venv/bin/activate
```

```
# or always use the venv's python directly:
```

```
.venv/bin/python -m cli.main --help
```

“.venv broken / weird import errors”

```
make clean      # nukes .venv + caches
make install    # rebuilds
```

“Port 7860 already in use” (Gradio web UI)

Another Gradio app is running. Kill it:

```
lsof -ti :7860 | xargs kill
```

Or change the port in web/app.py (server_port=7860).

CLI runs but no findings come back

1. Run `make doctor` — verify model is pulled and daemon is up.
2. Try a smaller file (≤ 30 lines).
3. Check `/tmp/ollama.log` for errors.
4. Lower `OLLAMA_NUM_CTX` to 4096 in `.env` if you’re hitting OOM (rare on 16 GB).

“LLM call failed twice” in logs

The model returned garbled output twice in a row. Causes: - Model was just loaded and crashed → retry the command. - Disk is full → `df -h`. - File contains binary blobs → our reader tolerates them with `errors="replace"`, but the prompt may still confuse the model. Try with a shorter file.

bandit / ruff are skipped

They live inside the venv. From outside the venv they’re not on PATH. The doctor displays them as SKIP but they still work when the engine is invoked via `.venv/bin/python` or `make`.

Reinstall if you’re sure they’re missing:

```
.venv/bin/pip install --force-reinstall bandit==1.7.10 ruff==0.6.9
```

“respx not installed” / test failures on import

```
.venv/bin/pip install -r requirements.txt
```

Resetting everything from scratch

```
make clean
rm -rf ~/.ollama/models
bash setup.sh
```

The last command re-pulls the model (~5 GB). Use only if you suspect a corrupted model file.

Performance is poor

- Confirm Metal is engaged: `top -pid $(pgrep ollama) -stats command,cpu,mem` — should show ~6 GB RSS and high CPU during inference.
- Check macOS Activity Monitor for thermal throttling.
- Close other heavy apps (Chrome, Docker) — they eat unified memory the model needs.
- Reduce `OLLAMA_NUM_CTX` in `.env` from 16384 to 8192.

Logs

- Ollama: `/tmp/ollama.log` (started by `setup.sh`’s `nohup ollama serve`)
- CodeSentinel: `stderr` — set `CODESENTINEL_LOG_LEVEL=DEBUG` for verbose output.

Getting help

- Open an issue on the GitHub repository.
- Check the model card: 04_model_card.md.
- Check known limitations: 08_limitations.md.